Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №16

Дисциплина: Основы программирования на Python

Тема: Разработка веб-приложения на Python с использованием Django

Вариант 2: Библиотека

Цель работы

Получить практические навыки разработки веб-приложений на Python с использованием фреймворка Django. В ходе работы необходимо познакомиться с архитектурой Django, создать проект и приложение, изучить структуру проекта, научиться создавать модели данных и использовать Django ORM, выполнять миграции, реализовывать связи между моделями, работать с QuerySet, использовать административную панель, создавать представления, применять URL-маршрутизацию и шаблоны, работать с формами и валидацией, реализовать CRUD-операции, аутентификацию, разграничение доступа, поиск, фильтрацию, сортировку, пагинацию и автоматические тесты.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать веб-приложение библиотеки с моделями Book, Author, Reader и Loan. Реализовать каталог книг, справочник авторов, выдачу книг читателям, возврат книг, поиск, список просроченных книг и страницу статистики. Приложение должно содержать не менее трёх моделей со связями, работать через Django ORM, поддерживать CRUD-операции, формы с валидацией, административную панель, поиск, фильтрацию, сортировку, пагинацию, аутентификацию и разграничение доступа, а также автоматические тесты.

Краткие теоретические сведения

Django — полнофункциональный веб-фреймворк, реализующий архитектурный шаблон MTV: Model описывает данные, Template отвечает за представление, View содержит логику обработки запроса. Проект (project) объединяет настройки и несколько приложений (app), каждое из которых решает отдельную задачу и содержит собственные модели, представления, маршруты и шаблоны.

Django ORM сопоставляет класс модели с таблицей базы данных. Поля объявляются как атрибуты класса, а связи задаются через ForeignKey, ManyToManyField и OneToOneField с параметром related_name для обратного доступа. Изменения моделей переносятся в базу механизмом миграций командами makemigrations и migrate, что позволяет развивать схему без ручного написания SQL.

QuerySet — ленивая коллекция объектов, к которой применимы фильтрация filter(), исключение exclude(), сортировка order_by(), срезы для пагинации и агрегаты aggregate() и annotate(). Методы select_related() и prefetch_related() заранее загружают связанные объекты и устраняют лишние запросы к базе.

Представления бывают функциональными и классовыми. Классы ListView, DetailView, CreateView, UpdateView и DeleteView реализуют типовые сценарии, требуя определить лишь модель и шаблон. URL-маршрутизация описывается в файле urls.py с помощью функции path() и пространства имён приложения.

Формы Django выполняют проверку данных на сервере: методы clean_<поле> проверяют отдельное поле, метод clean() — комбинацию полей. Административная панель автоматически строится по зарегистрированным моделям и настраивается классами ModelAdmin. Аутентификация встроена: декоратор login_required и миксин LoginRequiredMixin ограничивают доступ, а UserPassesTestMixin позволяет задать дополнительное условие. Middleware

обрабатывает запросы до и после представления, а CSRF-токен защищает формы от подделки запросов.

Проектирование базы данных

Модель	Поля	Связи
Genre	name (уникальное)	Многие ко многим с Book
Author	full_name (уникальное), country, birth_year	Один автор — много книг
Reader	full_name, email (уникальный), phone, registered_at	Один читатель — много выдач
Book	title, year, isbn (уникальный), price, copies_total, copies_available, description	FK на Author, M2M с Genre, много выдач
Loan	loan_date, due_date, return_date	FK на Book и Reader

Связь книги с жанрами реализована как «многие ко многим», поскольку одна книга может относиться к нескольким жанрам. Для модели Book заданы ограничения целостности на уровне базы: год издания в диапазоне от 1400 до 2100 и доступное количество экземпляров не больше общего. Для модели Loan задано ограничение, согласно которому срок возврата не может быть раньше даты выдачи. Свойства is_available и is_overdue вычисляют состояние книги и признак просрочки без хранения в базе.

Архитектура приложения

Запрос проходит путь от URL-маршрута к представлению, которое обращается к моделям через ORM, получает данные и передаёт их в шаблон. Формы обеспечивают валидацию вводимых данных, а административная панель использует те же модели для служебного редактирования.

Компонент	Назначение
config/settings.py, config/urls.py	Настройки проекта, подключение приложения library, маршруты верхнего уровня
library/models.py	Модели Genre, Author, Reader, Book, Loan с ограничениями и свойствами
library/views.py	Функциональные и классовые представления, миксин StaffRequiredMixin
library/forms.py	BookForm и LoanForm с проверкой ISBN, года и доступности экземпляров

Компонент	Назначение
library/admin.py	Настройка административной панели: списки, фильтры, встроенные выдачи
library/urls.py	Маршруты приложения с пространством имён library
templates/	Базовый шаблон, страницы каталога, книги, формы, статистики, читателей и входа
library/tests.py	Двадцать четыре теста страниц, CRUD-операций, выдач и моделей

Модели и Django ORM

Модели содержат русскоязычные подписи полей и метаданные: порядок сортировки по умолчанию, удобные для администратора названия в единственном и множественном числе. Метод __str__ задаёт читаемое представление объекта. В модели Book переопределён метод save(): если доступное количество экземпляров не задано или превышает общее, оно приводится к общему, что защищает от некорректного состояния при создании объекта через любые интерфейсы.

Листинг 16.1 — Модель книги (library/models.py)

```
class Book(models.Model):
    title = models.CharField("название", max_length=200, db_index=True)
    author = models.ForeignKey(Author, verbose_name="автор",
                               on_delete=models.CASCADE,
                               related_name="books")
    genres = models.ManyToManyField(Genre, verbose_name="жанры",
                                    related_name="books", blank=True)
    year = models.PositiveIntegerField(
        "год издания",
        validators=[MinValueValidator(1400), MaxValueValidator(2100)])
    isbn = models.CharField("ISBN", max_length=13, unique=True)
    price = models.DecimalField("цена", max_digits=10, decimal_places=2,
                                validators=[MinValueValidator(0)], default=0)
    copies_total = models.PositiveIntegerField("всего экземпляров", default=1)
    copies_available = models.PositiveIntegerField("доступно", default=1)


    class Meta:
        ordering = ["title"]
        constraints = [
            models.CheckConstraint(
                condition=models.Q(year__gte=1400, year__lte=2100),
                name="ck_book_year"),
            models.CheckConstraint(
                condition=models.Q(copies_available__lte=models.F("copies_total")),
                name="ck_book_copies"),
        ]


    @property
    def is_available(self) -> bool:
        return self.copies_available > 0
```

Представления и маршрутизация

Маршрут	Представление	Назначение
/	index	Главная страница со счётчиками и последними поступлениями
/books/	BookListView	Каталог с поиском, фильтрацией, сортировкой и пагинацией по десять записей
/books/<pk>/	BookDetailView	Карточка книги с жанрами и историей выдач
/books/add/	BookCreateView	Добавление книги, доступно сотрудникам
/books/<pk>/edit/	BookUpdateView	Редактирование книги, доступно сотрудникам
/books/<pk>/delete/	BookDeleteView	Удаление книги с подтверждением
/books/<pk>/borrow/	borrow_book	Выдача книги читателю по форме
/loans/<pk>/return/	return_loan	Возврат книги
/statistics/	statistics	Статистика фонда
/readers/	readers	Список читателей
/readers/<pk>/	reader_detail	Карточка читателя с выдачами
/accounts/login/	LoginView	Вход в систему

Каталог реализован классовым представлением BookListView с пагинацией по десять записей. В методе get_queryset() применяются select_related() и prefetch_related() для загрузки автора и жанров, поиск по названию, автору и ISBN через Q-объекты с оператором OR, фильтр по жанру и по наличию экземпляров, а также сортировка по выбранному полю с направлением. Словарь SORT_FIELDS сопоставляет имя сортировки полю ORM, что исключает передачу произвольного имени в запрос.

Листинг 16.2 — Фильтрация и сортировка каталога (library/views.py)

```
def get_queryset(self):
    queryset = Book.objects.select_related("author")
                           .prefetch_related("genres")
    query = self.request.GET.get("q", "").strip()
    genre = self.request.GET.get("genre", "")
    available = self.request.GET.get("available") == "1"
    sort = self.request.GET.get("sort", "title")
    order = self.request.GET.get("order", "asc")


    if query:
        queryset = queryset.filter(
            Q(title__icontains=query)
            | Q(author__full_name__icontains=query)
            | Q(isbn__icontains=query))
    if genre:
        queryset = queryset.filter(genres__name=genre)
    if available:
        queryset = queryset.filter(copies_available__gt=0)


    column = SORT_FIELDS.get(sort, "title")
```

```
        prefix = "-" if order == "desc" else ""
        return queryset.order_by(f"{prefix}{column}", "id")
```

Формы и валидация

Форма BookForm построена на ModelForm: поля и подписи берутся из модели, а дополнительные правила заданы методами clean_isbn() и clean(). Метод clean_isbn() проверяет, что ISBN состоит только из цифр и имеет длину десять или тринадцать символов. Метод clean() контролирует, что год издания не превышает текущий год и что доступное количество экземпляров не больше общего, добавляя ошибки к соответствующим полям.

Форма LoanForm используется при выдаче книги. В конструктор передаётся конкретная книга, поэтому проверка clean() может убедиться, что у неё есть свободные экземпляры, а метод clean_due_date() проверяет, что срок возврата позже даты выдачи. При успешной проверке представление уменьшает счётчик доступных экземпляров и сохраняет запись о выдаче, а при возврате увеличивает счётчик, ограничивая его общим количеством.

Административная панель

Все пять моделей зарегистрированы в административной панели. Для книги настроены отображение названия, автора, года и количества экземпляров, фильтры по жанрам и году, поиск по названию и ISBN, а также вычисляемый столбец доступности. Для авторов и жанров добавлены столбцы с числом связанных книг, для читателей — число выдач, для выдач — признак просрочки. Выдачи отображаются внутри карточки книги встроенным блоком TabularInline, что позволяет видеть историю непосредственно при редактировании книги.

Аутентификация и разграничение доступа

Просмотр каталога, карточек книг, читателей и статистики доступен без авторизации. Операции изменения данных требуют прав сотрудника: классы BookCreateView, BookUpdateView и BookDeleteView наследуют миксин StaffRequiredMixin, объединяющий LoginRequiredMixin и UserPassesTestMixin с

проверкой request.user.is_staff. Функции выдачи и возврата защищены декоратором staff_member_required. При попытке выполнить операцию без авторизации пользователь перенаправляется на страницу входа, а при недостаточных правах получает отказ в доступе. Демонстрационные пользователи создаются management-командой seed_demo.

Раздел тестирования

Тесты написаны на встроенном фреймворке Django TestCase и разделены на четыре группы. Базовый класс LibraryTestBase создаёт в setUpTestData общий набор данных: жанры, авторов, книги, читателей, выдачу и пользователей с разными правами.

Группа тестов	Кол-во	Что проверяется
PagesTests	12	Главная страница, каталог, вторая страница пагинации, карточка книги и страница 404, поиск по названию и автору, фильтр по жанру и наличию, сортировка, статистика, страницы читателей
CrudTests	6	Создание книги требует входа, запрещено обычному пользователю, разрешено сотруднику, ошибка валидации, изменение и удаление сотрудником
LoanTests	3	Выдача книги сотрудником с уменьшением счётчика, возврат с увеличением счётчика, запрет выдачи без прав
ModelTests	3	Связи между моделями, вычисление признака просрочки, доступность списка книг в административной панели

Все двадцать четыре теста проходят успешно. Дополнительно приложение проверялось вручную: выполнялись миграции, база заполнялась демонстрационными данными, в браузере проверялись каталог с поиском и сортировкой, карточка книги, создание и редактирование книги под учётной записью сотрудника, выдача и возврат книги, страница статистики и административная панель.

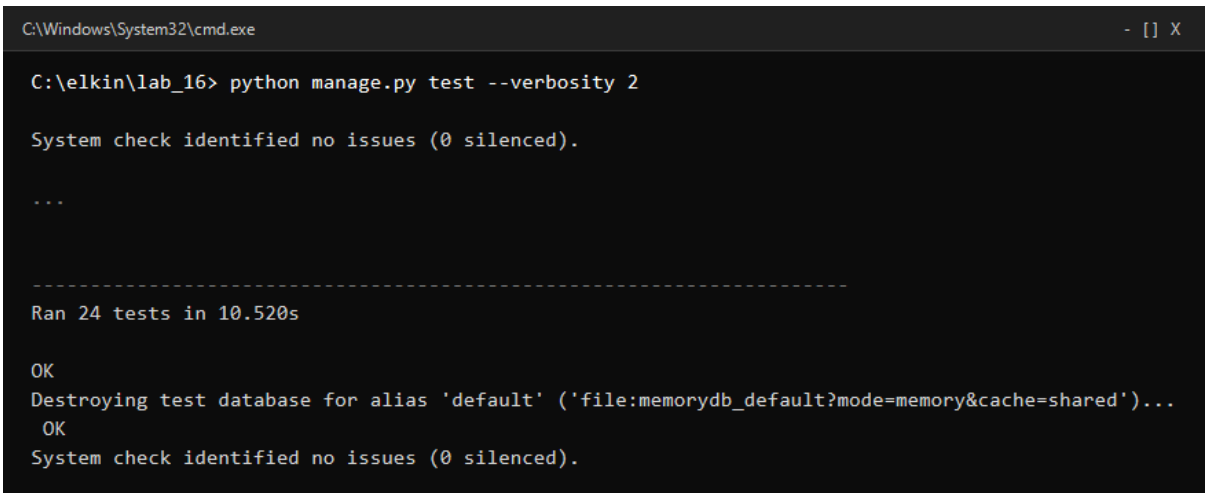


Рисунок 16.1 — Результат выполнения тестов Django

Листинги ключевых фрагментов программы

Листинг 16.3 — Выдача книги сотрудником (library/views.py)

```
@staff_member_required
def borrow_book(request, pk: int):
    book = get_object_or_404(Book, pk=pk)
    form = LoanForm(request.POST or None, book=book)

    if request.method == "POST" and form.is_valid():
        loan = form.save(commit=False)
        loan.book = book
        loan.loan_date = date.today()
        book.copies_available -= 1
        book.save()
        loan.save()
        messages.success(request,
                         f"Книга выдана: {loan.reader.full_name}")
        return redirect("library:book_detail", pk=book.pk)

    return render(request, "library/loan_form.html",
                  {"book": book, "form": form})
```

Листинг 16.4 — Статистика фонда (library/views.py)

```
def statistics(request):
    context = {
        "total_books": Book.objects.count(),
        "avg_price": Book.objects.aggregate(value=Avg("price"))["value"] or 0,
        "available_books": Book.objects.filter(copies_available__gt=0).count(),
        "overdue_loans": Loan.objects.filter(
            return_date__isnull=True, due_date__lt=date.today()).count(),
        "readers_count": Reader.objects.count(),
        "by_genre": Genre.objects.annotate(books_count=Count("books"))
                                 .order_by("-books_count"),
        "top_reader": Reader.objects.annotate(loans_count=Count("loans"))
                                    .order_by("-loans_count").first(),
        "expensive": Book.objects.order_by("-price").first(),
    }
    return render(request, "library/statistics.html", context)
```

Листинг 16.5 — Разграничение доступа (library/views.py)

```
class StaffRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self) -> bool:
        return self.request.user.is_staff


class BookCreateView(StaffRequiredMixin, CreateView):
    model = Book
    form_class = BookForm
    template_name = "library/book_form.html"
    success_url = reverse_lazy("library:book_list")
```

Вывод

В ходе выполнения лабораторной работы разработано веб-приложение библиотеки на фреймворке Django. Созданы пять моделей со связями «один ко многим» и «многие ко многим», ограничениями целостности и вычисляемыми свойствами; схема базы развивалась через миграции. Освоены Django ORM и QuerySet: фильтрация, поиск по нескольким полям, сортировка, пагинация и агрегаты с аннотированием. Реализованы функциональные и классовые представления, формы с серверной валидацией, административная панель со встроенными выдачами, аутентификация и разграничение доступа по признаку сотрудника. Двадцать четыре автоматических теста подтверждают корректность работы страниц, CRUD-операций, выдачи и возврата книг.